

DOCUMENT DE SPÉCIFICATION

Projet I1 – Jeu d'Échecs en Python

Module	I1 – Projet Informatique 2025/2026
Équipe	Équipe à compléter
Date	Mai 2026
Langage	Python 3 — Orienté Objet
Interface	Mode texte (terminal) + sauvegarde JSON
GitHub	https://github.com/enzonamj/echecs

1. Schéma des Classes — Diagramme UML

Le diagramme ci-dessous représente les classes définies dans le cahier des charges ainsi que leurs relations (héritage, composition, association). L'architecture respecte le principe d'**encapsulation** (attributs privés préfixés par `_`) et le principe d'**héritage** via la classe abstraite `Piece`.

Classe	Attributs principaux	Méthodes principales	Relation
Position	<code>_column</code> : str <code>_row</code> : int	<code>__str__()</code> <code>from_string()</code> <code>col_index</code>	Utilisée par <code>Piece</code> , <code>Board</code>
Piece (abstraite)	<code>_position</code> : Position <code>_color</code> : int	<code>isValidMove()</code> * <code>__str__()</code> *	Classe mère de <code>King</code> , <code>Queen</code> , <code>Bishop</code> , <code>Knight</code> , <code>Rook</code> , <code>Pawn</code>
King	<code>_position</code> <code>_color</code>	<code>isValidMove()</code> <code>__str__() → 'K'</code>	hérite de <code>Piece</code>
Queen	<code>_position</code> <code>_color</code>	<code>isValidMove()</code> <code>__str__() → 'Q'</code>	hérite de <code>Piece</code>
Bishop	<code>_position</code> <code>_color</code>	<code>isValidMove()</code> <code>__str__() → 'B'</code>	hérite de <code>Piece</code>
Knight	<code>_position</code> <code>_color</code>	<code>isValidMove()</code> <code>__str__() → 'N'</code>	hérite de <code>Piece</code>
Rook	<code>_position</code> <code>_color</code>	<code>isValidMove()</code> <code>__str__() → 'R'</code>	hérite de <code>Piece</code>
Pawn	<code>_position</code> <code>_color</code>	<code>isValidMove()</code> <code>__str__() → 'P'</code>	hérite de <code>Piece</code>

Classe	Attributs principaux	Méthodes principales	Relation
Board	<code>_squares : dict {Position: Piece}</code>	<code>getPiece() getPosition() movePiece() get_all_pieces() is_in_check() to_dict() / from_dict()</code>	Contient toutes les Piece
Player	<code>_name : str _color : int</code>	<code>askMove() color_name()</code>	Utilisé par Chess
AIPlayer	(hérite de Player)	<code>askMove() [redéfini] get_random_valid_move()</code>	hérite de Player
Chess	<code>board : Board players : list[Player] currentPlayer : Player move_history : list</code>	<code>initPlayers() displayBoard() isValidMove() updateBoard() switchPlayer() isCheckMate() isStalemate() save() / load() play()</code>	Orchestre le jeu (agrège Board et Players)

* méthodes abstraites — doivent être implémentées par chaque sous-classe

Relations entre classes

Héritage (is-a)	King, Queen, Bishop, Knight, Rook, Pawn héritent de Piece. AIPlayer hérite de Player.
Composition (has-a)	Chess possède un Board et une liste de Players. Board possède un dictionnaire de Pieces.
Dépendance (uses)	Piece utilise Position et Board dans isValidMove(). Player utilise Position pour parser ses coups.

2. Organisation de l'Équipe

2.1 Répartition des tâches

Chaque membre de l'équipe est responsable d'au moins un type de pièce (méthode `isValidMove`) ainsi que des tests unitaires associés. Voici la répartition proposée :

Membre	Pièce(s) assignée(s)	Autres responsabilités
Membre 1	Roi (King) Reine (Queen)	Classe Position Classe Board Tests Position + Board
Membre 2	Tour (Rook) Fou (Bishop)	Classe Chess (boucle principale) Tests Rook + Bishop
Membre 3	Cavalier (Knight) Pion (Pawn)	Classe Player + AIPlayer Tests Knight + Pawn
Membre 4	Relecture globale	Sauvegarde / Chargement JSON Tests d'intégration Diagramme UML

2.2 Outils et méthode de travail

- **GitHub** — Dépôt public pour le partage et la gestion des versions du code. Chaque membre travaille sur sa propre branche et ouvre une Pull Request avant de fusionner dans main.
- **Discord / WhatsApp** — Communication instantanée entre les membres de l'équipe pour les questions rapides et les mises à jour de progression.
- **unittest (Python)** — Chaque classe est accompagnée de tests unitaires pour vérifier son comportement avant l'intégration dans le programme principal.
- **Moodle** — Dépôt des livrables (document de spécification, lien GitHub) aux dates prévues par l'intervenant.

2.3 Gestion de projet

Un chef de projet **tournant** sera désigné à chaque séance. Son rôle est de vérifier l'avancement des tâches, identifier les blocages et s'assurer que tout le monde a bien compris ce qu'il doit faire. Cette rotation permet à chacun de développer des compétences en coordination d'équipe.

Les développements ne démarreront qu'à partir de la **séance 3**, après la réunion de démarrage, pour que tout le monde parte sur les mêmes bases. On commence par les versions simplifiées (`isCheckMate()` retourne False, `isValidMove()` retourne True), puis on affine séance après séance.

3. Améliorations Envisagées

Voici les améliorations que l'équipe envisage d'implémenter, classées par priorité (de la plus accessible à la plus ambitieuse) :

Priorité	Amélioration	Description	Impact
1 — Haute	Détection de l'echec et du mat	Implémenter <code>is_in_check()</code> et <code>isCheckMate()</code> de façon cohérente avec simulation de coups.	5 points +points
2 — Haute	IA aléatoire (AIPlayer)	L'IA choisit un coup valide aleatoire parmi toutes ses pieces.	4 points +points
3 — Moyenne	Interface graphique (pygame)	Affichage graphique de l'echiquier avec des pieces visuelles.	4 points +points
4 — Moyenne	Déplacements spéciaux	Roque (petit et grand) et prise en passant — respecter les règles officielles des échecs.	3 points +points
5 — Basse	Sauvegarde en base de données	Remplacer le fichier JSON par une base de données SQL.	2 points +points
6 — Basse	Promotion des pions	Quand un pion atteint la derniere rangée, le joueur choisit une nouvelle piece le promouvant.	2 points +points

3.1 Détail de l'interface graphique (pygame)

L'amélioration la plus visible est l'interface graphique avec **pygame**. Le plateau sera affiché avec des cases colorées (marron/beige), les pièces représentées par des lettres ou des images PNG. Le joueur clique sur une pièce puis sur la case de destination pour jouer. Les coups invalides seront ignorés et un message s'affichera. L'affichage se mettra à jour après chaque coup.

Si le temps manque, on garde le mode texte (terminal) qui est déjà fonctionnel et qui répond pleinement au cahier des charges.

3.2 IA améliorée

Dans un premier temps, l'IA jouera de façon **aléatoire** (elle choisit un coup valide au hasard parmi toutes ses pièces). Dans un second temps, si on a le temps, on peut améliorer l'IA en lui donnant une **évaluation simple** de la position : par exemple, capturer une pièce adverse plutôt que de jouer au hasard.